

King Saud University
College of Computer and Information Sciences
Department of Information Technology
Second Semester 1447H – Fall 2026
IT462 – Big Data Systems in IT

Big Data Analysis of Online Retail Transactions Using Apache Spark

Phase 6 :Report

Group#5 members		
Student name	Student ID	Email
Maha Alnassar	444200604	444200604@student.ksu.edu.sa
Lujain Alhusan	444200785	444200785@student.ksu.edu.sa
Raseel Almanea	444201169	444201169@student.ksu.edu.sa

Date of submission: Sunday, May 10, 2026

Supervised by
Dr. Ameera Almasoud

1 Table of Contents

1 Introduction	4	
1.1 Background and Context		4
1.2 Problem Statement		4
1.3 Objective	5	
1.4 Scope and Limitations		5
1.4.1 Scope	5	
1.4.2 Limitations	5	
2 Data Selection and Understanding		5
2.1 Dataset Description	5	
2.2 Structure and Schema		5
2.3 Initial Data Quality Observation		6
2.4 Ethical and Legal Consideration		6
3 Data Preprocessing	6	
3.1 Data Cleaning	6	
3.2 Reduction	7	
3.3 Transformation	8	
3.4 Summary of Preprocessing Pipeline		8
4 RDD Operations	9	
4.1 RDD Design	9	
4.2 Why RDDs Were Useful Beyond DataFrames and SQL		9
4.3 Key Transformations and Actions	9	
4.3.1 Block 1 – Customer Feature Extraction and Churn Candidate Identification		9
4.3.2 Block 2 – Country-Level Churn Risk Aggregation		10
4.3.3 Block 3 – Enriched Customer Profiling via Join, Risk Scoring, and Output		11
4.3.4 Block 4 – Customer Base Discovery via flatMap and distinct		12
4.4 Reflection on RDD Usage	13	
5 SQL Operations	13	
5.1 DataFrame and View Creation		13
5.2 Key Analytical Queries	14	
5.3 Summary of Insights from SQL		16
6 Machine Learning Operations		16
6.1 Problem Formulation	16	
6.2 Feature Pipeline	16	
6.3 Model Choice	17	
6.4 Training and Evaluation Setup		17
6.5 Results	18	
6.6 Interpretation	19	
6.7 Limitations	19	
7 References	19	

Table Of Figure

Figure 1: Customer-level RDD structure used to store RFM features for distributed processing in Apache Spark.	9
Figure 2: Scala map transformation used to extract customer RFM features into an RDD structure.	10
Figure 3: RDD aggregation and churn metric calculation by country.	10
Figure 4: RDD-based customer risk labeling and temporal feature join	11
Figure 5: flatMap and distinct transformations for extracting unique customer IDs.....	12
Figure 5-1: Figure X: Registering the DataFrame as a temporary SQL view for analytical queries.	13
Figure 5-2: Query 1: Country Value Profile	14
Figure 5-3: Q1_sample output	14
Figure 5-4: Query 2: High-Value Inactive Customers.....	14
Figure 5-5: Q2_sample output	14
Figure 5-6: Query 3: Inactive Customers.....	14
Figure 5-7: Q3_sample output	15
Figure 5-8: Query 4: Churn Percentage by Country	15
Figure 5-9: Q4_sample output	15
Figure 5-10: Query 5: Ranked Risk Customers Using Window Functions.....	15
Figure 5-11: Q5_sample output	15
Figure 5-12: Query 6: Country Risk and Statistical Summary	16
Figure 5-13: Q6_sample output	16
Figure 6-1: Removing unnecessary attributes and selecting the RFM features for clustering.	17
Figure 6-2: Selecting the best K value using the Silhouette Score.	17
Figure 6-3: Train/test split using a fixed random seed (42).	17
Figure 6-4: K-Means clustering results and customer cluster profiles.	18
Figure 6-5: Gaussian Mixture Model clustering results and cluster profiles.	19
Figure 6-6: Comparison between K-Means and Gaussian Mixture Model performance.	19

Table Of Table

Table 1 :Main Attributes and Their Types.....	6
Table 2 Before and after cleaning	7
Table 3 Quantitative Impact.....	7
Table 4: Top 5 most inactive churn-risk customers sorted by recency in descending order.	10
Table 5: Country-level churn risk summary by customer count, average frequency, and churn ratio.	11
Table 6: Sample enriched customer profiles after joining behavioral and temporal RDDs.	12
Table 7: Sample of unique customer IDs extracted using flatMap and distinct transformations.	12
Table 8: best clustering performance	18
Table 9: Cluster Profiles (K-Means).....	18

1 Introduction

This project analyzes customer purchasing behavior using the Online Retail II dataset to identify meaningful customer segments and detect potential churn-risk customers. By applying RFM-based features, Spark SQL analysis, RDD operations, and unsupervised machine learning techniques, the project aims to transform raw transactional data into useful business insights that can support customer retention and marketing decision-making.

1.1 Background and Context

In the era of big data, organizations generate large volumes of transactional data from customer interactions, particularly in domains such as e-commerce and retail. Analyzing this data is essential for understanding customer behavior, improving decision-making, and enhancing business performance. Customer segmentation is a fundamental analytical technique that groups customers based on shared characteristics, enabling targeted marketing and personalized services. It has been widely recognized as a key driver for improving customer retention, optimizing marketing strategies, and increasing profitability [1].

A commonly used approach for representing customer behavior is the RFM (Recency, Frequency, Monetary) model, which captures how recently a customer made a purchase, how often they purchase, and how much they spend. The RFM model is widely adopted in marketing analytics due to its effectiveness in identifying valuable and loyal customer segments [2].

However, in many real-world datasets, there are no predefined labels indicating customer segments, making supervised learning approaches unsuitable. As a result, unsupervised learning techniques such as clustering are commonly applied to automatically discover hidden patterns and group customers based on similarity in their behavior [3]. Among these techniques, K-Means is widely used due to its simplicity and efficiency [4], while Gaussian Mixture Models (GMM) provide a probabilistic framework that allows for more flexible cluster shapes [5]. These methods enable organizations to extract actionable insights from raw data and support data-driven decision-making.

1.2 Problem Statement

This project addresses a customer behavior analysis problem using the Online Retail II dataset. The main goal is to analyze purchasing behavior, discover meaningful customer segments, and identify customers who may be at risk of churn. Since the dataset does not contain predefined labels such as loyal, inactive, high-value, or churn-risk customers, unsupervised machine learning techniques, specifically K-Means and Gaussian Mixture Models, were used to group customers based on similarities in their behavior.

The analysis is based on the RFM model, which includes **RecencyDays**, **Frequency**, and **Monetary** value. These features help measure how recently a customer purchased, how often they purchase, and how much they spend. Customers with high RecencyDays and low Frequency were considered potential churn-risk customers, while customers with high Monetary values were analyzed as important customers because losing them could negatively affect revenue.

This problem is important because businesses often have large amounts of transactional data but need clearer insights into customer loyalty, inactivity, and churn risk. The results can help marketing teams, business managers, and decision-makers understand customer behavior, identify high-risk or valuable customers, compare customer patterns across countries, and design targeted retention strategies. Spark SQL and RDD

operations were used to support the analysis by summarizing customer behavior, calculating churn indicators, and extracting customer-level features for deeper interpretation.

1.3 Objective

- Clean and preprocess the retail transaction dataset.
- Build RFM features to represent customer behavior.
- Analyze customer patterns using Spark RDD and SQL.
- Apply K-Means and GMM clustering models.
- Evaluate clusters using silhouette score.
- Interpret customer segments and extract business insights.

1.4 Scope and Limitations

1.4.1 Scope

This project analyzes customer purchasing behavior using the Online Retail II dataset from **2009 to 2011**. It focuses on RFM features: RecencyDays, Frequency, and Monetary value. The scope includes data cleaning, preprocessing, feature engineering, Spark SQL analysis, RDD operations, and clustering using K-Means and Gaussian Mixture Models.

The project also identifies churn-risk customers, high-value customers, and country-level churn patterns. The results could later be extended into a hosted analytics dashboard for business users.

1.4.2 Limitations

The dataset does not include actual labels for customer groups or churn status, so the clustering results cannot be fully validated against real categories. The analysis mainly depends on RFM features and does not include customer demographics, satisfaction, or marketing data.

Also, the data is historical, covering only 2009 to 2011, so it may not reflect current customer behavior. Hosting and real-time deployment are outside the current scope and would require a backend system, database, user interface, and periodic model retraining.

2 Data Selection and Understanding

2.1 Dataset Description

- **Source and Domain:** The **Online Retail II** dataset belongs to the **e-commerce and retail analytics** domain. It is available from **Kaggle / UCI Machine Learning Repository** at: <https://www.kaggle.com/datasets/mashlyn/online-retail-ii-uci>.
- **Dataset Size:** The dataset consists of a single CSV file (*online_retail_II.csv*) with a size of approximately 94.85 MB. It contains about 1,067,371 rows and 8 columns.
- **Time Coverage:** The dataset covers a period from December 2009 to December 2011.

2.2 Structure and Schema

The Online Retail II dataset contains transactional records from a UK-based online retail store. The main attributes included in the dataset are as follows:

Table 1 :Main Attributes and Their Types

Attribute	Description	Type	Example Value
-----------	-------------	------	---------------

InvoiceNo	Unique invoice number assigned to each transaction	Nominal/Categorical (String)	489434
StockCode	Unique product/item identifier	Nominal/Categorical (String)	85048
Description	Product name and description	Nominal / Categorical	15CM CHRISTMAS GLASS
Quantity	Number of units purchased in each transaction	Numeric (Integer)	12
InvoiceDate	Date and time when the transaction occurred	Temporal (Datetime)	2009-12-01 07:45:00
UnitPrice	Price per unit in GBP	Numeric (Float)	6.95
CustomerID	Unique identifier for each customer	Nominal/Categorical (String)	13085.0
Country	Country where the customer is located	Nominal/Categorical (String)	United Kingdom

This dataset includes a mix of numeric, categorical, temporal, and text attributes, which enables diverse analytical and predictive tasks such as customer behavior analysis, sales trend analysis, and revenue modeling. The dataset does not provide formal primary or foreign keys. However, **InvoiceNo**, **StockCode**, and **CustomerID** can be considered natural identifier fields for transactions, products, and customers respectively.

2.3 Initial Data Quality Observation

The dataset inspection showed several data quality issues that needed preprocessing. Missing Customer IDs were removed because they are required for customer-level analysis, while missing product descriptions were replaced with “Unknown.” Invalid records with non-positive quantities or prices were filtered out, duplicates were removed, and extreme numerical outliers were detected using the IQR method. These cleaning steps improved the dataset’s accuracy, consistency, and reliability for analysis and modeling.

2.4 Ethical and Legal Consideration

The dataset is publicly available and suitable for academic use. It does not include direct personal identifiers, but it contains a **CustomerID**, which makes it **pseudonymized rather than fully anonymized** because customer behavior can still be tracked across transactions. The analysis was conducted ethically by focusing on aggregated insights and avoiding any attempt to re-identify individual customers.

3 Data Preprocessing

3.1 Data Cleaning

The data cleaning process was performed to improve data quality and ensure the dataset is suitable for customer behavior analysis using the RFM model. The cleaning phase addressed missing values, invalid records, duplicate entries, and outliers.

1. **Missing Values:**The dataset contained **occasional missing values** in key fields:
 - **CustomerID (critical):** Rows were **removed** since customer identification is required for RFM aggregation
 - **Description (non-critical):** Missing values replaced with **"Unknown"**.

After this step the remaining rows is **82,436** and removed rows is **243,007**.

2. **Invalid Values:** Transactions with: **Quantity ≤ 0 , UnitPrice ≤ 0** . Were removed, as they represent returns or incorrect entries. So rows removed is **18,815** and remaining rows is **80,549**.
3. **Duplicate Records:** Duplicate transactions were removed to avoid overcounting purchases. So duplicates removed is **26,124** and final dataset size is **779,425**.
4. **Outlier Detection:** Outliers in the **Quantity** column were identified using the **IQR method**, which is suitable for skewed transactional data. Extreme values were handled to reduce their impact on analysis.

Table 2 Before and after cleaning

Issue	Before	After
Missing CustomerID rows	243,007	0
Missing Description	4382	Replace with “Unknown”
Invalid Quantity/Price	18,815	0
Duplicate records	26,124	0
Outliers	51,119	0
Final dataset size	1,067,371	728,306

3.2 Reduction

The data reduction process was applied to decrease dataset complexity while preserving essential information for customer behavior analysis. This included feature selection and aggregation to a customer-level dataset.

- **Feature Selection:** The original dataset contained **8 attributes**. During feature selection, irrelevant and redundant attributes were removed to reduce dimensionality and improve analysis efficiency. **Description** and **StockCode** attributes were excluded as they are **not required for RFM analysis**, since they do not contribute to customer-level behavioral metrics. The remaining selected attributes were: InvoiceNo, Quantity, InvoiceDate, UnitPrice, CustomerID, Country. This reduced the number of columns from **8 to 6**, ensuring that only relevant features were retained for aggregation and RFM computation.
- **Aggregation (Transaction To Customer Level):** Since the dataset originally contained **transaction-level records**, it was aggregated to **customer-level data** by grouping on: CustomerID and Country. The following RFM metrics were computed:
 - **Recency:** days since last purchase
 - **Frequency:** number of distinct invoices
 - **Monetary:** total spending

Additionally, the **timestamp of the most recent purchase** was retained. This transformation converted multiple transaction records into **one summarized record per customer**, significantly reducing dataset size.

Table 3 Quantitative Impact

Metric	Before	After
Number of columns	8	6
Columns	invoiceNo, StockCode, Description, Quantity, InvoiceDate, UnitPrice, CustomerID, Country	InvoiceNo, Quantity, InvoiceDate, UnitPrice, CustomerID, Country

Data level	Transaction-level	Customer-level
Distinct customers	5,630	5,630
Rows	Large (transaction records)	Significantly reduced (1 row per customer)

3.3 Transformation

The dataset was transformed to ensure compatibility with machine learning models and to enhance the representation of customer behavior patterns.

- **Type Conversions:** Several columns were converted to appropriate data types:
 1. **LastPurchaseTS** : converted from string to **timestamp format**.
 2. **Recency, Frequency, Monetary** : cast to **numeric types**

These conversions ensured consistency, enabled time-based calculations, and prevented processing errors.

- **Feature Engineering:** New features were derived from the timestamp to capture temporal purchasing behavior:
 1. **PurchaseHour** : hour of transaction , **PurchaseDay** : day of the week, **PurchaseMonth** : month of transaction.
 2. **IsWeekend** : binary indicator (weekend vs weekday)

These features provide insights into customer purchasing patterns and temporal trends.

- **Encoding of Categorical Variables:** The **Country** column was encoded using **StringIndexer**, converting categorical values into numerical indices. This step allows machine learning models to process categorical data effectively.
- **Feature Vector Construction:** All numerical and encoded features were combined into a single feature vector using **VectorAssembler**. This unified representation simplifies model input and ensures compatibility with Spark ML pipelines.
- **Scaling / Normalization:** Feature scaling was applied using **StandardScaler** to normalize numerical features. This method standardizes features by: centering around the mean and scaling by standard deviation. Scaling ensures that all features contribute equally during model training and prevents bias due to different value ranges.

Overall, the transformation process improved data consistency, enabled advanced feature representation, and prepared the dataset for efficient machine learning modeling.

3.4 Summary of Preprocessing Pipeline

The preprocessing pipeline followed a structured sequence to prepare the dataset for analysis and modeling:

Raw Data → Cleaning → Reduction → Transformation → Final Dataset for ML

- **Raw Data:** Original transactional dataset loaded from CSV into Spark
- **Cleaning:** Handled missing values, removed invalid records, eliminated duplicates, and detected outliers
- **Reduction:** Selected relevant features and aggregated transaction-level data into customer-level (RFM) metrics

- **Transformation:** Applied type conversions, feature engineering, encoding, and scaling
- **Final Dataset:** Structured customer-level dataset ready for machine learning tasks

The code exist in GitHub as present in the Appendix and file name RetailPreprocessing.scala

4 RDD Operations

4.1 RDD Design

After preprocessing the Online Retail II dataset using Spark DataFrames and SQL, the customer-level data was converted into an RDD for fine-grained processing. Each record in the RDD represents a single customer aggregated from multiple transactions, following the structure below:

```
// Core customer RDD type
RDD[(String, String, Int, Int, Double)]
// Fields: (CustomerID, Country, RecencyDays, Frequency, Monetary)

// Example record
val customerFeatures = rows.map(row => {
  val customerID = row(0)
  val country    = row(1)
  val recency    = row(2).toInt
  val frequency  = row(3).toInt
  val monetary   = row(4).toDouble
  (customerID, country, recency, frequency, monetary)
})
// Type: MapPartitionsRDD[String, String, Int, Int, Double]
```

Figure 1: Customer-level RDD structure used to store RFM features for distributed processing in Apache Spark.

4.2 Why RDDs Were Useful Beyond DataFrames and SQL

While Spark DataFrames and SQL provided efficient structured querying during the preprocessing and aggregation phases, RDDs offered several advantages for the churn analysis tasks in this phase:

1. Fine-grained key-value operations: The `reduceByKey` transformation allowed aggregating churn metrics at the country level with full control over the accumulation function, avoiding the overhead of converting back to a DataFrame for a `GroupBy`.
2. Flexible joins without a schema: Combining the behavioral RDD with the temporal RDD via `join` was straightforward using keyed tuples, without the need to define or align a schema - useful when the two data sources have structurally different shapes.
3. Explicit risk scoring formula: Computing a composite churn-exposure score per customer required a custom mathematical formula that does not map cleanly to a single SQL expression; the RDD `reduce` action allowed this to be computed in one pass.
4. Controlled output with `PrintWriter`: Saving the enriched output to a formatted text file was done using Java's `PrintWriter` inside a `foreachPartition`-style loop, giving precise control over the output format that Spark's default save methods do not easily provide.

4.3 Key Transformations and Actions

4.3.1 Block 1 – Customer Feature Extraction and Churn Candidate Identification

- **Purpose:** Extract the core RFM features from the preprocessed dataset and identify customers who are candidates for churn based on behavioral thresholds.
- **Inputs:** A DataFrame converted to an RDD via `.rdd`, containing the customer-level dataset produced at the end of Phase 2 preprocessing. Each Row holds: CustomerID (String), Country (String), LastPurchaseTS (Timestamp), RecencyDays (Int), Frequency (Int), Monetary (Double), PurchaseHour (Int), PurchaseMonth (Int), PurchaseDay (Int), IsWeekend (Int).
- **Operations used:**

1. map – extract and type-cast RFM fields from each DataFrame row
2. filter – retain only records where RecencyDays > 90 AND Frequency < 5
3. count – quantify high-value customers within the churn-risk subset
4. take – retrieve the five most inactive churn-risk customers after sortBy

- Scala snippet:

```
// Step 1: map - extract RFM features
val customerFeatures = rows.map(row => {
  val customerID = row(0)
  val country    = row(1)
  val recency    = row(3).toInt
  val frequency  = row(4).toInt
  val monetary   = row(5).toDouble
  (customerID, country, recency, frequency, monetary)
})
```

Figure 2: Scala map transformation used to extract customer RFM features into an RDD structure.

- Sample output:

CustomerID	Country	RecencyDays	Frequency	Monetary
13526	United Kingdom	738	2	346.48
17056	United Kingdom	738	1	128.60
12636	USA	738	1	141.00
14654	United Kingdom	738	1	216.62
17592	United Kingdom	738	1	148.30

Table 4: Top 5 most inactive churn-risk customers sorted by recency in descending order.

- Interpretation: The filter results show that 808 customers who spent more than 500 have not purchased for over 90 days and transacted fewer than five times. The sortBy/take output reveals that the most inactive customers have a recency of 738 days, nearly the full span of the dataset, confirming complete disengagement.

4.3.2 Block 2 – Country-Level Churn Risk Aggregation

- Purpose: Aggregate churn-related indicators by country to understand geographic distribution of churn risk, average customer engagement, and spending patterns per region.
- Inputs::customerRDD - the mapped RDD from Block 1 of type RDD[(String, String, Int, Int, Double)] representing (CustomerID, Country, RecencyDays, Frequency, Monetary).
- Operations used:
 1. map – transform each customer record into a keyed pair (country -> churn indicator tuple)
 2. reduceByKey – sum churn counts and accumulate frequency and monetary totals per country
 3. mapValues – compute derived metrics (average frequency, average monetary, churn ratio)
- Scala snippet:

```
// Step 1: map - key by country with churn indicators
val countryStats = customerRDD.map(c => {
  val h = if(c.recencyDays>90 && c.frequency<2 && c.monetary<300) 1 else 0
  (c.country, (1, h, c.frequency, c.monetary))
}).reduceByKey((a, b) =>
  (a._1+b._1, a._2+b._2, a._3+b._3, a._4+b._4)
)

// Step 2: mapValues - derive averages and churn ratio
val countrySummary = countryStats.mapValues {
  case (c, h, f, m) =>
    (c, h, f.toDouble/c, m/c, h.toDouble/c)
}

countrySummary.take(5).foreach(println)
```

Figure 3: RDD aggregation and churn metric calculation by country.

- Sample output:

Country	Customers	High-Risk	Avg Freq	Avg Monetary	Churn Ratio
Australia	15	5	5.53	2112.92	0.333
Portugal	24	4	3.83	1929.97	0.167

Country	Customers	High-Risk	Avg Freq	Avg Monetary	Churn Ratio
United Kingdom	5227	970	5.97	1732.93	0.186
Brazil	2	1	1.90	935.0	0.500
Canada	4	2	1.75	991.34	0.500

Table 5: Country-level churn risk summary by customer count, average frequency, and churn ratio.

- **Interpretation:** This block answers the question: Which countries exhibit the highest concentration of churn risk, and how do spending patterns vary by geography? Australia shows a high average monetary value (2,112) with a churn ratio of 33%, indicating high-value but disengaged customers. Brazil and Canada each show a 50% churn ratio, although their customer counts are very small. The United Kingdom has the lowest churn ratio (18.6%) but contributes the largest number of at-risk customers (970) because of its dominant customer base.

4.3.3 Block 3 – Enriched Customer Profiling via Join, Risk Scoring, and Output

- **Purpose:** Combine behavioral RFM features with temporal purchasing patterns through an RDD join, compute a composite churn-exposure score per customer using a custom formula, and persist the enriched profiles to disk.
- **Inputs:** Two RDDs derived from the same customerRDD source: (1) customerRiskRDD of type RDD[(Int, (Int, Int, Double, String))] - keyed by CustomerID, holding (RecencyDays, Frequency, Monetary, RiskLabel); (2) temporalRDD of type RDD[(Int, (Int, Int, Int, Int))] - keyed by CustomerID, holding (PurchaseMonth, PurchaseHour, IsWeekend). Both contain 5,630 records and share CustomerID as the join key.
- **Operations used:**
 1. map – build customerRiskRDD with risk labels and temporalRDD with time features
 2. join – merge both RDDs on CustomerID to create enriched profiles
 3. map – apply composite risk-scoring formula to each enriched record
 4. reduce – sum all individual scores into a total churn-exposure metric
 5. save (PrintWriter) – write enriched records to output.txt
- **Scala snippet:**

```
// Step 1: build risk-labeled RDD
val customerRiskRDD = customerRDD.map(c => {
  val l = if(c.recencyDays>90 && c.frequency<2 && c.monetary<300) "High Risk"
  else if(c.recencyDays>60 && c.frequency<0) "Medium Risk"
  else "Low Risk"
  (c.customerID, (c.recencyDays, c.frequency, c.monetary, l))
})

// Step 2: temporal RDD
val temporalRDD = customerRDD.map(c => {
  (c.customerID, (c.purchaseMonth, c.purchaseHour, c.isWeekend))
})

// Step 3: join on CustomerID
val enriched = customerRiskRDD.join(temporalRDD)
```

Figure 4: RDD-based customer risk labeling and temporal feature join

- **Sample output (enriched records):**

CustomerID	Recency	Freq	Monetary	Risk	Month	Hour	Weekend
16408	591	1	231.93	High Risk	4	14	0
17158	32	5	1890.00	Low Risk	11	15	0
14882	26	11	1511.65	Low Risk	11	12	0
13298	1	90	76.00	Low Risk	9	11	0
13278	40	8	1270.97	Low Risk	10	11	0

Table 6: Sample enriched customer profiles after joining behavioral and temporal RDDs.

- **Interpretation:** This block answers the question: What is the combined behavioral and temporal profile of each customer, and what is the overall churn exposure? High Risk customers show high recency and low purchase frequency, while Low Risk customers show more active purchasing behavior. The total churn-exposure score of 55,687.82 summarizes the overall portfolio risk.

4.3.4 Block 4 – Customer Base Discovery via flatMap and distinct

- **Purpose:** Extract all CustomerIDs from raw transactional records and deduplicate them to determine the true size of the active customer base, independent of the aggregated DataFrame.\
- **Inputs:** A raw text RDD loaded directly from the CSV file using `sc.textFile()`, where each element is a comma-separated string representing one transaction record. This RDD reflects the full transaction-level dataset (728,306 rows after cleaning) before any aggregation. Each line contains fields in the order: CustomerID, Country, RecencyDays, Frequency, Monetary, LastPurchaseTS, PurchaseHour, PurchaseMonth, PurchaseDay, IsWeekend.
- **Operations used:**
 1. `flatMap` – extract CustomerID from every transaction line, handling edge cases
 2. `distinct` – remove duplicate IDs to obtain unique customers
 3. `first` – verify dataset loading by displaying the first raw record
 4. `collect` – safely retrieve a small sample to validate the pipeline
- **Scala snippet:**

```
// flatMap: extract CustomerID from each raw line
val customersFlat = rdd.flatMap(line => {
  val cols = line.split(",")
  if (cols.length > 0) List(cols(0))
  else List()
})

// distinct: deduplicate
val uniqueCustomers = customersFlat.distinct()
```

Figure 5: flatMap and distinct transformations for extracting unique customer IDs

- **Sample output:**

Unique Customer IDs (sample)
15488
14515
17749
17422
14216
15978
13237
17378
13170
14463

Table 7: Sample of unique customer IDs extracted using flatMap and distinct transformations.

- **Interpretation:** This block answers the question: How many distinct customers exist in the dataset, and is the raw data structured correctly? The first action confirmed that records contain all expected fields (CustomerID, Country, RFM metrics, timestamp, temporal features). The flatMap and distinct pipeline independently validated the customer count obtained from the DataFrame aggregation phase.

The use of `collect` on a bounded sample rather than the full RDD is an important scalability practice - collecting the entire RDD to the driver would risk an out-of-memory error for datasets of this size.

4.4 Reflection on RDD Usage

Performance and Scalability Observations

Several performance considerations were encountered during the RDD-based analysis:

1. Lazy evaluation: All transformations (map, filter, flatMap, reduceByKey, join) are lazy in Spark - they do not execute until an action is triggered. This allowed the entire transformation pipeline to be planned and optimized by the Spark scheduler before execution, reducing redundant data passes.[7].
2. No caching was applied in this phase because each RDD was consumed by a single action chain. For iterative pipelines, persisting the feature RDD with `rdd.persist(StorageLevel.MEMORY_AND_DISK)` would avoid recomputing all transformations from scratch on each iteration.[8].
3. The `sortBy` operation before `take(5)` triggers a full sort across all partitions, which is relatively expensive for this use case. For production pipelines, a top-k heap approach or a bounded sort would be more efficient than sorting the entire RDD before taking five records.[8].

Limitations and Difficulties

Working with RDDs presented several challenges compared to the DataFrame and SQL phases:

1. Type safety and debugging: RDDs use generic tuple types (e.g., `RDD[(String, (Int, Int, Double, String))]`), which makes it easy to introduce off-by-one index errors when accessing nested tuple fields. Debugging required careful use of `take(1).foreach(println)` checkpoints between transformations to verify intermediate structures.[9]
2. No schema introspection: Unlike DataFrames, RDDs do not carry column names. This made the code harder to read and maintain, particularly in the join step where the nested tuple structure became deeply nested.[9]
3. Join correctness: The RDD join required that both RDDs be keyed on the same data type (Int for CustomerID). A mismatch in key types between `customerRiskRDD` and `temporalRDD` would produce an empty result without any error message, making such bugs difficult to detect without explicit validation.[9]

5 SQL Operations

5.1 DataFrame and View Creation

The preprocessed customer transaction dataset was loaded into Apache Spark using the `SparkSession` API. The CSV file was imported with header detection and automatic schema inference enabled to correctly identify data types such as integers, doubles, and timestamps. The dataset included customer behavioral attributes related to the RFM model, including `RecencyDays`, `Frequency`, and `Monetary`, in addition to customer location and purchase timing information. After loading the dataset, the DataFrame schema and sample rows were displayed to verify that the preprocessing stage was successful and that all required features were correctly formatted. The DataFrame was then registered as a temporary SQL view to allow SQL-based analytical operations using Spark SQL.

The temporary SQL view used throughout the analysis was:

```
df.createOrReplaceTempView("customers")
```

Figure 5-1: Figure X: Registering the DataFrame as a temporary SQL view for analytical queries.

This view enabled the execution of multiple analytical SQL queries related to customer segmentation, churn analysis, and customer value profiling.

5.2 Key Analytical Queries

- **Query 1: Country Value Profile**

Business Question: Which countries contain the highest-value customers based on RFM behavior?

SQL Query:

```
SELECT
    Country,
    COUNT(DISTINCT CustomerID) AS total_customers,
    ROUND(AVG(RecencyDays), 2) AS avg_recency,
    ROUND(AVG(Frequency), 2) AS avg_frequency,
    ROUND(AVG(Monetary), 2) AS avg_monetary,
    SUM(CASE WHEN Monetary > 500 THEN 1 ELSE 0 END) AS high_value_customers
FROM customers
GROUP BY Country
HAVING COUNT(DISTINCT CustomerID) >= 5
ORDER BY avg_monetary DESC, avg_frequency DESC
```

Figure 5-2: Query 1: Country Value Profile

Sample Output:

Country	total_customers	avg_recency	avg_frequency	avg_monetary	high_value_customers
EIRE	5	307.0	107.2	67016.7	3
Norway	13	185.62	3.46	3455.37	10
Greece	5	236.2	3.6	3384.92	5

Figure 5-3: Q1_sample output

Interpretation: This query reveals substantial differences in customer purchasing behavior across countries. EIRE showed exceptionally high monetary values and purchase frequency, indicating the presence of highly valuable customers. These findings suggest that customer value is geographically distributed and may require country-specific marketing strategies.

- **Query 2: High-Value Inactive Customers**

Business Question: Which customers are considered high value but show signs of inactivity and churn risk?

SQL Query:

```
SELECT
    CustomerID,
    Country,
    RecencyDays,
    Frequency,
    Monetary,
    CASE
        WHEN RecencyDays > 180 AND Frequency < 5 AND Monetary > 500 THEN 'Critical Risk'
        WHEN RecencyDays > 90 AND Frequency < 5 AND Monetary > 500 THEN 'High Risk'
        WHEN RecencyDays > 90 AND Frequency < 8 THEN 'Moderate Risk'
        ELSE 'Lower Risk'
    END AS risk_segment
FROM customers
WHERE Monetary > 500
ORDER BY RecencyDays DESC, Monetary DESC
LIMIT 20
```

Figure 5-4: Query 2: High-Value Inactive Customers

Sample Output:

CustomerID	Country	RecencyDays	Frequency	Monetary	risk_segment
13204	United Kingdom	731	1	966.66	Critical Risk
13768	United Kingdom	726	2	875.97	Critical Risk
14061	United Kingdom	724	2	553.43	Critical Risk

Figure 5-5: Q2_sample output

Interpretation: This query identifies customers who previously generated relatively high revenue but have not interacted with the business recently. These customers represent potential churn-risk customers and may benefit from targeted retention campaigns or personalized promotions.

- **Query 3: Inactive Customers**

Business Question: Which customers demonstrate low engagement and inactive purchasing behavior?

SQL Query:

```
SELECT CustomerID, Country, RecencyDays, Frequency, Monetary
FROM customers
WHERE RecencyDays > 90 AND Frequency < 5
ORDER BY RecencyDays DESC
```

Figure 5-6: Query 3: Inactive Customers

Sample Output:

CustomerID	Country	RecencyDays	Frequency	Monetary
13526	United Kingdom	738	2	346.48
17056	United Kingdom	738	1	128.6
12636	USA	738	1	141.0

Figure 5-7: Q3_sample output

Interpretation: The query identifies inactive customers with very high recency values and low purchasing frequency. These customers are likely disengaged and represent a significant churn-risk segment within the dataset.

- **Query 4: Churn Percentage by Country**

Business Question: Which countries have the highest proportion of churn-risk customers?

SQL Query:

```
SELECT
    Country,
    COUNT(*) AS total_customers,
    SUM(CASE WHEN RecencyDays > 90 AND Frequency < 5 THEN 1 ELSE 0 END) AS churn_customers,
    ROUND(
        SUM(CASE WHEN RecencyDays > 90 AND Frequency < 5 THEN 1 ELSE 0 END) * 100.0 / COUNT(*),
        2
    ) AS churn_percentage
FROM customers
GROUP BY Country
ORDER BY churn_percentage DESC
```

Figure 5-8: Query 4: Churn Percentage by Country

Sample Output:

Country	total_customers	churn_customers	churn_percentage
Nigeria	1	1	100.00
European Community	1	1	100.00
West Indies	1	1	100.00

Figure 5-9: Q4_sample output

Interpretation: This analysis highlights countries where customer inactivity is proportionally high. Although some countries contain only a few customers, the results still reveal significant differences in customer engagement patterns across geographic regions.

- **Query 5: Ranked Risk Customers Using Window Functions**

Business Question: Who is the top churn-risk customers within each country?

SQL Query:

```
WITH risk_segmented AS (
    SELECT
        CustomerID,
        Country,
        RecencyDays,
        Frequency,
        Monetary,
        CASE
            WHEN RecencyDays > 90 AND Frequency <= 2 AND Monetary > 300 THEN 'Critical Risk'
            WHEN RecencyDays > 60 AND Frequency <= 4 THEN 'Moderate Risk'
            ELSE 'Low Risk'
        END AS risk_segment
    FROM customers
),
ranked_customers AS (
    SELECT
        CustomerID,
        Country,
        RecencyDays,
        Frequency,
        Monetary,
        risk_segment,
        DENSE_RANK() OVER(PARTITION BY Country ORDER BY Monetary DESC) AS rank_in_country
    FROM risk_segmented
    WHERE risk_segment IN ('Critical Risk', 'Moderate Risk')
)
SELECT *
FROM ranked_customers
WHERE rank_in_country <= 3
ORDER BY Country, rank_in_country
```

Figure 5-10: Query 5: Ranked Risk Customers Using Window Functions

Sample Output:

CustomerID	Country	RecencyDays	Frequency	Monetary	risk_segment	rank_in_country
12424	Australia	162	4	1812.75	Moderate Risk	1
12389	Australia	402	3	1229.33	Moderate Risk	2
12434	Australia	84	3	1103.43	Moderate Risk	3

Figure 5-11: Q5_sample output

Interpretation: This query uses CTEs and window functions to rank high-risk customers within each country according to monetary value. It enables organizations to prioritize retention efforts toward the most financially valuable customers.

- **Query 6: Country Risk and Statistical Summary**

Business Question: What are the statistical characteristics of customer risk behavior across countries?

SQL Query:

```
WITH risk_flag AS (
    SELECT
        Country,
        CustomerID,
        RecencyDays,
        Frequency,
        Monetary,
        CASE
            WHEN RecencyDays > 90 AND Frequency <= 2 AND Monetary > 300 THEN 1
            ELSE 0
        END AS high_risk_flag
    FROM customers
),
country_analysis AS (
    SELECT
        Country,
        COUNT(*) AS total_customers,
        SUM(high_risk_flag) AS high_risk_customers,
        ROUND(100.0 * SUM(high_risk_flag) / COUNT(*), 2) AS churn_percentage,
        ROUND(SUM(Frequency), 2) AS avg_frequency,
        ROUND(SUM(Monetary), 2) AS avg_monetary,
        ROUND(SUM(Monetary * Monetary), 2) AS median_monetary,
        ROUND(VAR(SUM(Monetary), 2)) AS monetary_variance
    FROM risk_flag
    GROUP BY Country
    HAVING COUNT(*) >= 10
)
SELECT *
FROM country_analysis
ORDER BY churn_percentage DESC, avg_monetary DESC
```

Figure 5-12: Query 6: Country Risk and Statistical Summary

Sample Output:

Country	total_customers	high_risk_customers	churn_percentage	avg_recency	avg_frequency	avg_monetary	median_monetary	monetary_variance
Sweden	18	9	50.00	298.61	4.5	1271.55	908.05	1103644.72
Channel Islands	10	4	40.00	228.2	4.4	3345.3	1606.31	1.104183873E7
Norway	13	5	38.46	185.62	3.46	3455.37	1821.91	1.828050304E7

Figure 5-13: Q6_sample output

Interpretation: This query combines statistical aggregation with churn analysis to provide a deeper understanding of customer behavior. Countries with high monetary variance indicate highly inconsistent spending patterns, which may complicate segmentation and prediction tasks.

5.3 Summary of Insights from SQL

- High-value customers with long inactivity periods were identified as critical churn-risk segments.
- Customer purchasing behavior varied significantly across countries in terms of recency, spending, and frequency.
- Several countries showed high churn percentages, indicating potential regional retention issues.
- Monetary values demonstrated high variance in some countries, suggesting diverse customer behavior patterns.
- These findings supported the use of clustering techniques based primarily on the RFM model features.

6 Machine Learning Operations

6.1 Problem Formulation

This project uses **unsupervised clustering** to identify hidden customer segments because the dataset has no predefined labels. Customers are grouped using **RFM features**: RecencyDays, Frequency, and Monetary value, which help measure customer engagement, loyalty, and value. The analysis also aims to detect inactive and churn-risk customers for targeted business actions. To improve clustering quality, extreme monetary outliers above **10,000** were removed during preprocessing.

6.2 Feature Pipeline

The Spark ML pipeline included the following stages:

1. **VectorAssembler** :Combined the RFM attributes into a single feature vector.
2. **StandardScaler** :Standardized feature values to prevent larger numerical attributes from dominating the clustering process.

The selected features were:

```
val colsToDrop = Seq(
  "CustomerID",
  "LastPurchaseTS",
  "Country",
  "CountryIndex"
).filter(ml_df.columns.contains)

val cleanDF0 = ml_df.drop(colsToDrop: _*)

// =====
// Keep Required Columns
// =====
val requiredCols = Seq(
  "RecencyDays",
  "Frequency",
  "Monetary"
).filter(cleanDF0.columns.contains)

val cleanDF = cleanDF0.na.drop(requiredCols)
  .filter(col("Monetary") < 10000)
```

Figure 6-1: Removing unnecessary attributes and selecting the RFM features for clustering.

The following attributes were excluded:

- **CustomerID** : identifier only
- **LastPurchaseTS** : timestamp not directly useful
- **Country** : categorical features are not required for clustering
- **CountryIndex** : encoded geographic identifier without direct behavioral meaning

The preprocessing pipeline was fitted only on the training dataset to avoid data leakage.

6.3 Model Choice

Two clustering models were used: K-Means , and Gaussian Mixture Model (GMM) .

- **K-Means**: K-Means was selected because it is computationally efficient and highly suitable for numerical customer segmentation tasks. It is also easy to interpret and commonly used in RFM-based clustering problems.
- **Gaussian Mixture Model**: GMM was selected because it supports probabilistic clustering and can model overlapping customer behaviors more effectively than K-Means.

Several values of K ranging from 2 to 6 were tested for K-Means. The optimal value of K was selected based on the highest silhouette score.

The best result was achieved using:

```
K = 2, Silhouette Score = 0.7232
K = 3, Silhouette Score = 0.6814
K = 4, Silhouette Score = 0.6684
K = 5, Silhouette Score = 0.6054
K = 6, Silhouette Score = 0.5154
Best K: 2
```

Figure 6-2: Selecting the best K value using the Silhouette Score.

6.4 Training and Evaluation Setup

The dataset was split into: 70% training data and 30% testing data using a fixed random seed:

```
val Array(trainDF, testDF) =
  cleanDF.randomSplit(Array(0.7, 0.3), seed = 42)
```

Figure 6-3: Train/test split using a fixed random seed (42).

The preprocessing pipeline was trained only on the training set to prevent information leakage into the testing data. Since clustering is an unsupervised learning task, class imbalance handling was not applicable.

Evaluation Metric: The primary evaluation metric used was the Silhouette Score. This metric measures how well data points fit within their assigned cluster compared to other clusters. Higher silhouette values indicate stronger separation and better clustering quality.

6.5 Results

The clustering models were evaluated using the Silhouette Score to measure cluster quality and separation. Based on the experimental evaluation discussed previously, **K = 2** achieved the best clustering performance for the K-Means model.

Table 8: best clustering performance

Model	Best K	Silhouette Score	Notes
K-Means	2	0.7232	Best performing model

Gaussian Mixture Model	2	0.6419	Lower clustering quality
------------------------	---	--------	--------------------------

The experimental results showed that the K-Means model achieved better clustering performance compared to the Gaussian Mixture Model, obtaining the **highest silhouette score of 0.7232**. In contrast, the Gaussian Mixture Model achieved a **lower silhouette score of 0.6419**, indicating weaker cluster separation.

The generated clusters successfully separated inactive low-value customers from loyal high-value customers based on the RFM features. Therefore, K-Means was selected as the final clustering model for customer segmentation.

Example Cluster Profiles (K-Means)

Table 9: Cluster Profiles (K-Means)

Cluster	Avg Recency	Avg Frequency	Avg Monetary	Interpretation
Cluster 0	237.54	3.02	757.78	Inactive low-value customers
Cluster 1	45.19	14.08	4580.57	Loyal high-value customers

The cluster profiling results indicate that customers in Cluster 1 had lower recency values and significantly higher frequency and monetary values, representing loyal and highly engaged customers. Meanwhile, Cluster 0 mainly contained inactive customers with lower spending behavior and a higher likelihood of churn.

```

=====
K-Means Model with K = 2
=====
Silhouette Score: 0.7232

Cluster Distribution:
+-----+-----+
|prediction|count|
+-----+-----+
|      0| 1357|
|      1|   245|
+-----+-----+

Cluster Profiling:
+-----+-----+-----+-----+-----+
|prediction|customers|avg_recency|avg_frequency|avg_monetary|
+-----+-----+-----+-----+-----+
|0|      1357|237.53868828297715|3.0198968312453944|757.7819086219597|
|1|      245|45.19183673469388|14.077551020408164|4580.57288163265|
+-----+-----+-----+-----+-----+

K-Means Score Saved: 0.7232

```

Figure 6-4: K-Means clustering results and customer cluster profiles.

```

=====
Gaussian Mixture Model with K = 2
=====
Silhouette Score: 0.6419

Cluster Distribution:
+-----+-----+
|prediction|count|
+-----+-----+
|      0| 361|
|      1|1241|
+-----+-----+

Cluster Profiling:
+-----+-----+-----+-----+-----+
|prediction|customers|avg_recency|avg_frequency|avg_monetary|
+-----+-----+-----+-----+-----+
|0|      361|53.79224376731302|11.645429362880886|3828.789279778389|
|1|     1241|253.01611603545527|2.693795326349718|619.166929895245|
+-----+-----+-----+-----+-----+

GMM Weights:
[0.2490354650757543, 0.7509645349242245]

```

Figure 6-5: Gaussian Mixture Model clustering results and cluster profiles.

```

=====
SUMMARY COMPARISON
=====
Best K-Means Silhouette Score: 0.7232
Gaussian Mixture Silhouette Score: 0.6419
Best Model: K-Means

```

Figure 6-6: Comparison between K-Means and Gaussian Mixture Model performance.

6.6 Interpretation

The clustering results demonstrated that RecencyDays and Frequency played a major role in separating inactive customers from highly engaged customers. Customers with: low recency , high frequency and high monetary value .Were grouped into a loyal customer segment, while customers with high recency and low frequency formed a churn-risk segment. These findings align strongly with expected business behavior and confirm the effectiveness of the RFM model for customer segmentation.

Error Analysis

Some overlap may occur among medium-value customers because their behavior is not clearly distinct. This may be due to limited features, missing demographic data, behavioral variation, and remaining noise in the transactions.

6.7 Limitations

Methodological Limitations

- Only two clustering algorithms were evaluated.
- Hyperparameter tuning was limited.
- No dimensionality reduction methods such as PCA were applied.

Data Limitations

- The dataset lacked demographic and product-level information.
- Temporal behavioral evolution was not deeply modeled.
- Some countries contained very small customer populations, which may reduce statistical reliability.

Future Improvements

- Include additional behavioral and demographic attributes.
- Apply advanced clustering approaches such as DBSCAN or hierarchical clustering.
- Perform deeper hyperparameter optimization and feature engineering.

7 References

- [1] P. Kotler and K. L. Keller, *Marketing Management*, 15th ed. Pearson, 2016.
- [2] A. M. Hughes, *Strategic Database Marketing*. Chicago, IL, USA: Probus Publishing, 1994.
- [3] J. Han, M. Kamber, and J. Pei, *Data Mining: Concepts and Techniques*, 3rd ed. Morgan Kaufmann, 2011.
- [4] J. MacQueen, "Some methods for classification and analysis of multivariate observations," in *Proc. 5th Berkeley Symp. Mathematical Statistics and Probability*, 1967, pp. 281–297.
- [5] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.
- [6] P. J. Rousseeuw, "Silhouettes: A graphical aid to the interpretation and validation of cluster analysis," *Journal of Computational and Applied Mathematics*, vol. 20, pp. 53–65, 1987.
- [7] M. Zaharia et al., "Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing," *USENIX NSDI*, 2012.
- [8] H. Karau and R. Warren, *High Performance Spark*. O'Reilly Media, 2017.
- [9] B. Chambers and M. Zaharia, *Spark: The Definitive Guide*. O'Reilly Media, 2018.

8 Appendix : GitHub

GitHub Link : <https://github.com/Raseealmanea/OnlineRetailTransactions.git>